

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1522
---------------------	---	---------------------	---------

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I **MAGESH. S (192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting realtime analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for timeseries and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Q1. Website Clicks, Mobile Interactions & Transactions Platform

A platform collecting billions of website clicks, mobile interactions, and customer transactions daily has almost certainly been built around a Lambda-style architecture combining a distributed message bus, a data lake, and separate batch/stream processing layers, since no single component could handle both the ingestion volume and the mixed structured/semi-structured data at this scale.

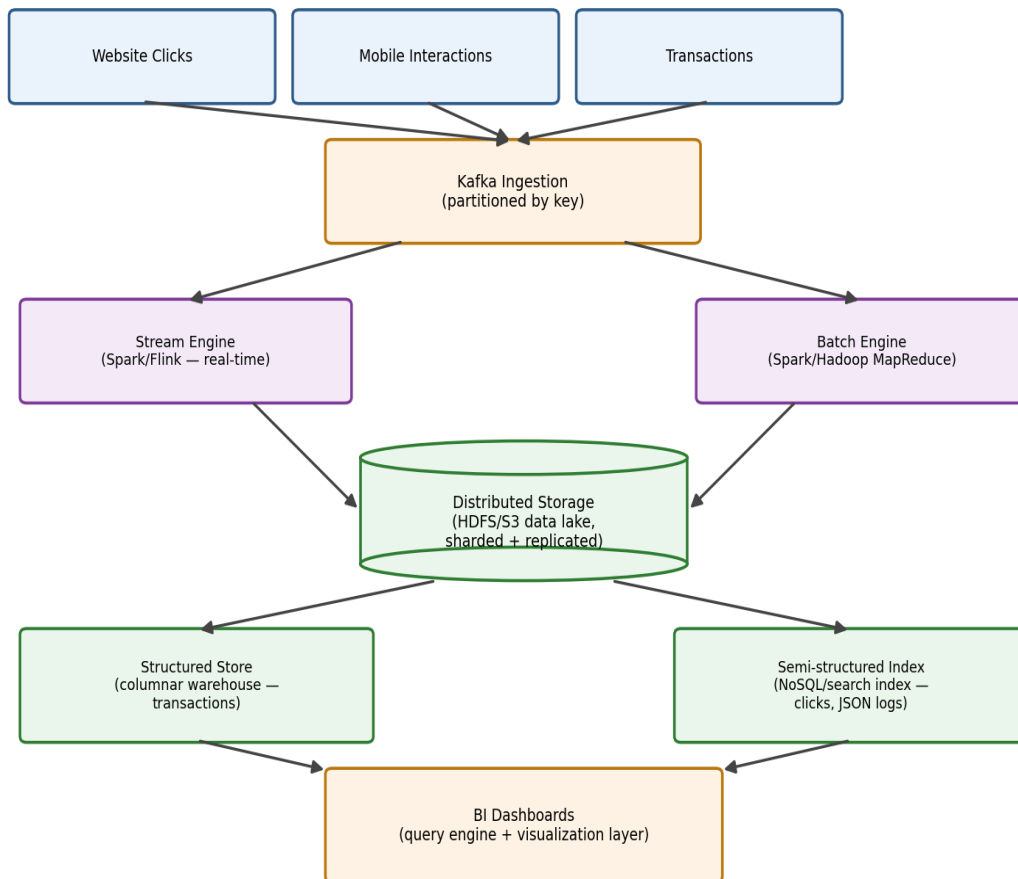
Probable ingestion framework and distributed storage

- Ingestion: Apache Kafka (or a managed equivalent like AWS Kinesis) is the most likely ingestion backbone — it is the de facto standard for absorbing billions of daily events with partitioned, ordered, durable delivery.
- Distributed storage: a data lake on HDFS or cloud object storage (S3/ADLS) most likely underlies the platform, since it can store both structured transaction records and semi-structured click/interaction logs cheaply at massive scale without forcing a rigid schema upfront.
- Batch processing: Apache Spark (or legacy Hadoop MapReduce for older systems) is the likely batch engine, chosen for its ability to process huge historical datasets for reporting and model training.

Inferring how structured and semi-structured data are separated and indexed

Transaction data (well-defined schema: order ID, amount, timestamp) is likely loaded into a columnar data warehouse (e.g., a Hive/Parquet-based warehouse or a cloud warehouse like BigQuery/Redshift) for fast structured queries. Click and interaction data (variable-schema JSON events) is likely stored in a NoSQL store or indexed via a search engine (e.g., Elasticsearch) so that free-form fields can still be efficiently queried without a rigid schema — this separation is the standard evidence-based pattern for platforms handling both data types at scale.

Q1 — Reverse-Engineered Clickstream/Transaction Platform



Likely stream-processing engines for real-time analytics

Spark Streaming or Apache Flink is the most probable real-time engine, sitting alongside the batch path and consuming the same Kafka topics — this dual-path (batch + streaming) design is the classic Lambda architecture pattern, chosen so that dashboards can show both up-to-the-minute metrics and deep historical trends without one workload blocking the other.

Scalability and partitioning strategy

Kafka topics are almost certainly partitioned by a key such as user ID or session ID, so related events land on the same partition (preserving order per user) while spreading total load across many brokers. The data lake would similarly be sharded/replicated across nodes, and the warehouse tables likely partitioned by date for efficient time-range queries — this combination is the standard way such platforms scale horizontally rather than relying on a single powerful server.

How dashboards and BI systems likely connect

A distributed SQL query engine (e.g., Presto/Trino or the warehouse's native query layer) most likely sits between the storage layer and BI tools (Tableau, Power BI, Looker), letting analysts query billions of rows without scanning raw files directly — dashboards would refresh from pre-aggregated tables for speed rather than querying raw event streams live.

Q2. E-commerce Cart, Checkout & Payment Tracking Platform

An e-commerce platform tracking cart additions, abandoned checkouts, product views, and payment behavior across regions has likely been built with an event-driven architecture where every user action is emitted as a discrete event, since this is the only practical way to reconstruct a session-level view for real-time personalization across distributed regional traffic.

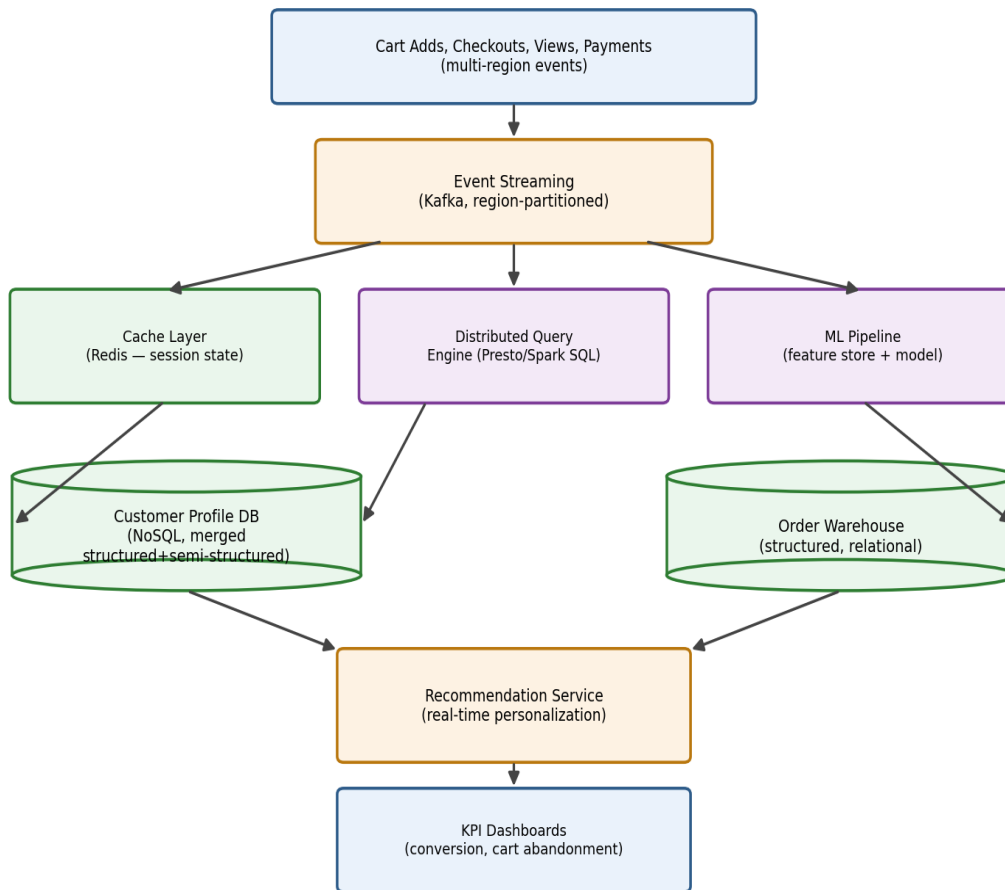
Likely event streaming, profiling database, and recommendation workflow

- Event streaming: Kafka (region-partitioned topics) is the probable backbone, allowing each region's traffic to be processed close to where it's generated while still feeding a global pipeline.
- Customer profiling database: a NoSQL database (e.g., DynamoDB, Cassandra, or MongoDB) most likely stores merged customer profiles, since it can flexibly combine structured order history with semi-structured browsing behavior under one customer ID without rigid schema migrations.
- Recommendation workflow: a feature store feeding a trained ML model (likely collaborative filtering or a learned embedding model) probably generates recommendations, refreshed periodically from batch-computed features and refined with real-time signals.

Inferring session/clickstream/transaction synchronization

Given the requirement for real-time personalization, session data is likely correlated using a session or user ID propagated through every event, with a caching layer (most probably Redis) holding the current session state for sub-millisecond lookups during a live browsing session, while the full clickstream and transaction history are asynchronously written to durable storage for later analysis.

Q2 — Reverse-Engineered E-commerce Personalization Pipeline



Probable caching systems, query engines, and ML stages

- Caching: Redis or Memcached for session state and frequently accessed product/recommendation data.
- Distributed query engine: Presto/Spark SQL likely powers ad hoc analytical queries across the merged dataset without needing to move data into a separate system first.
- ML stages: an offline feature engineering and training stage (batch, e.g., nightly) followed by a real-time serving stage that combines the trained model with the user's live session signals.

Merging structured order data and semi-structured browsing data

Structured order records (from a relational order-management system) are most likely joined with semi-structured browsing logs (JSON clickstream events) at the analytics layer via a common customer/session key, materialized into a unified view (e.g., a wide table or document) that downstream models and dashboards can query without repeatedly performing expensive joins.

Scalability and fault-tolerance decisions

Given multi-region operation, the platform likely uses regional Kafka clusters with cross-region replication for disaster recovery, horizontally scalable stateless application servers behind load balancers, and a database layer designed for eventual consistency across regions (favoring availability during network partitions) since strict global consistency would be too slow for real-time personalization at this scale.

How dashboards and KPI monitoring are generated

KPIs like conversion rate and cart abandonment are most likely computed by scheduled batch jobs aggregating the event stream into summary tables, which BI dashboards then query — with a smaller set of critical real-time KPIs (e.g., current site traffic) likely served directly from the streaming layer for immediate visibility.

Q3. Healthcare Platform: Patient Histories, Diagnostic Images, Wearables & Prescriptions

A healthcare platform combining patient histories, diagnostic images, wearable sensor streams, and prescription records has almost certainly been built as a hybrid database architecture, since no single database type can efficiently handle both highly structured records (prescriptions, patient demographics) and large unstructured binary data (diagnostic images) with the strict compliance requirements healthcare demands.

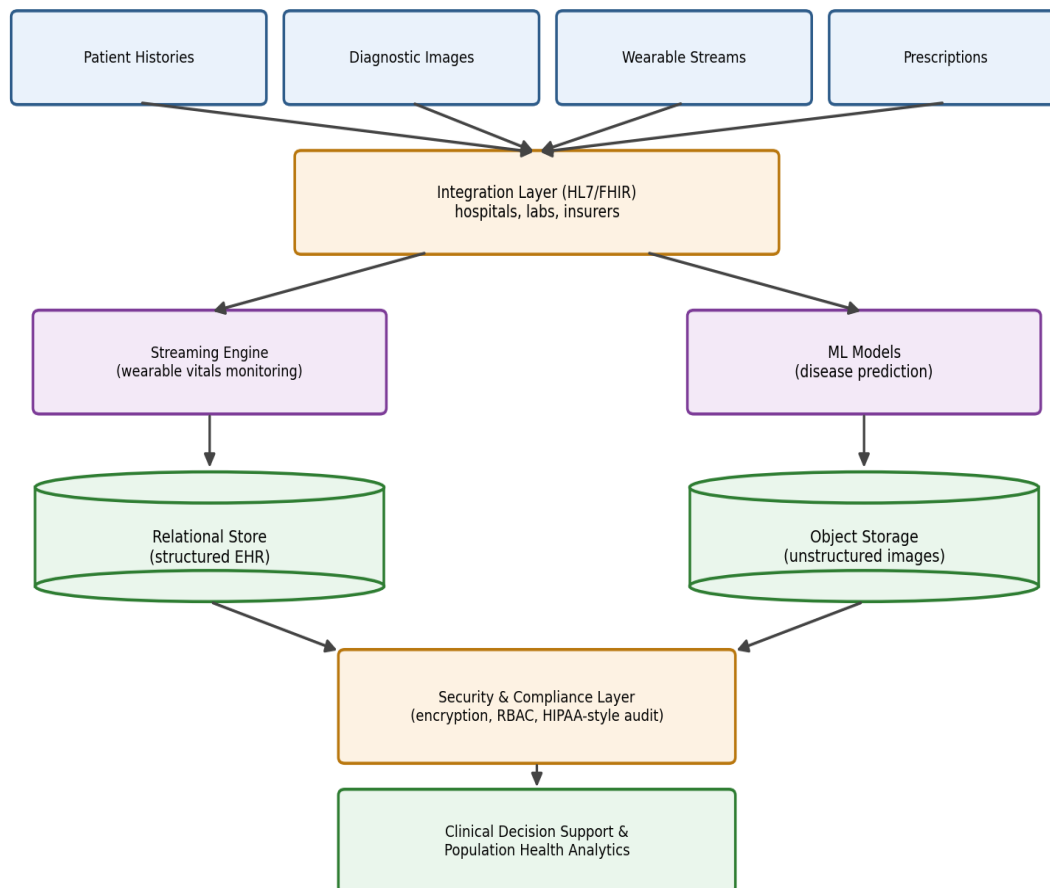
Probable hybrid database architecture

- Structured data (patient demographics, visit history, prescriptions) is most likely stored in a relational database (e.g., PostgreSQL/Oracle) or a structured EHR-specific system, since this data has well-defined relationships and requires transactional integrity.
- Unstructured data (diagnostic images — X-rays, MRIs) is most likely stored in object storage (e.g., a PACS system backed by cloud object storage), which is optimized for large binary files rather than a relational database.

Likely data integration tools connecting hospitals, labs, and insurers

Given the multi-organization nature of this data, an integration layer built on HL7 or the newer FHIR standard is highly probable, since these are the industry-standard interoperability protocols specifically designed to let hospitals, laboratories, and insurers exchange patient data in a common format despite running different internal systems.

Q3 — Reverse-Engineered Healthcare Big Data Platform



Streaming and ML frameworks for disease prediction and monitoring

Wearable sensor streams (heart rate, glucose, activity) most likely flow through a streaming engine (Spark Streaming or Flink) for continuous vitals monitoring and anomaly alerting, while separately trained ML models (likely classification or survival-analysis models trained in batch on historical patient data) probably support disease-risk prediction, with predictions periodically refreshed rather than computed per-event given the complexity of clinical models.

Privacy, encryption, and compliance enforcement

Given the sensitivity of health data, the platform almost certainly enforces field- and disk-level encryption (at rest and in transit), role-based access control scoped to clinical need-to-know, and detailed audit logging of every access to patient records — required under regulations like HIPAA (or regional equivalents) — with likely de-identification/anonymization applied before any data is used for broader population health research.

Analytics pipeline for clinical decision-making and population health

Individual clinical decision support most likely runs as near-real-time inference against the streaming/ML layer (e.g., flagging an at-risk patient to a clinician), while population health studies most likely run as separate batch analytics jobs against the aggregated, de-identified data warehouse — this split reflects the very different latency and privacy requirements of the two use cases.

Q4. Smart City Traffic, CCTV, Weather & Transport Platform

A smart city platform continuously gathering traffic sensor feeds, CCTV metadata, weather updates, and public transport activity has most likely been built with an edge-to-cloud architecture, since transmitting all raw sensor and video data to a central location continuously would be both bandwidth-prohibitive and too slow for real-time congestion response.

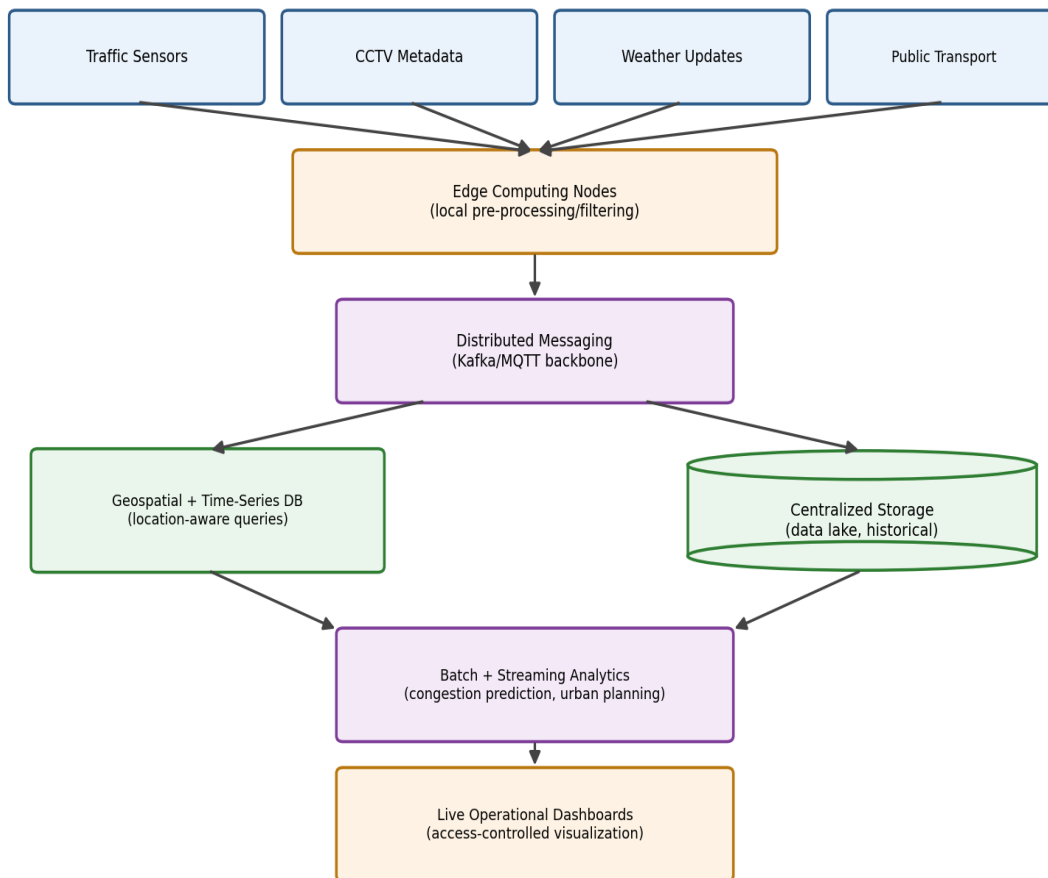
Probable edge computing, messaging, and centralized storage design

- Edge computing: local edge nodes at intersections/transit hubs most likely pre-process raw feeds (e.g., extracting vehicle counts from video rather than sending raw footage) before transmission, reducing bandwidth and enabling faster local reactions.
- Distributed messaging: a Kafka or MQTT-based messaging backbone is highly probable for the lightweight, high-frequency sensor and metadata messages typical of IoT/smart-city deployments.
- Centralized storage: a cloud data lake likely aggregates historical data from all edge sources for city-wide analysis and long-term planning.

Inferring geospatial analytics and streaming for congestion prediction

Congestion prediction most likely combines a geospatial/time-series database (storing sensor readings tagged with location and timestamp) with a streaming analytics layer that continuously recomputes traffic-flow estimates as new sensor events arrive — this is the standard pattern since congestion state changes continuously and location context is essential to every query.

Q4 — Reverse-Engineered Smart City Data Platform



Likely databases for time-series and location-aware queries

A time-series database (e.g., InfluxDB/TimescaleDB) most probably handles high-frequency sensor readings efficiently, while a geospatial-capable database or extension (e.g., PostGIS) likely supports location-based queries (e.g., "congestion within 2km of this intersection") — a general-purpose relational database without these extensions would struggle at this query pattern and scale.

Combining batch and real-time analytics for urban planning

Real-time streaming analytics most likely drive immediate operational decisions (e.g., adaptive traffic signal timing), while separate batch analytics over the accumulated historical data most likely support longer-term urban planning insights (e.g., where to build new transit routes) — reflecting the different time horizons of operational versus strategic decisions.

Security and access-control measures

Given that CCTV metadata and citizen movement data are sensitive, the platform most likely applies strict role-based access control (separating operational traffic-management access from any citizen-identifiable data), anonymization of personally identifiable elements in CCTV metadata, and encrypted transmission from edge devices to the cloud to prevent tampering or interception of infrastructure control signals.

How visualization systems likely deliver live intelligence

An operations-center dashboard, most likely built on a real-time visualization tool connected directly to the streaming analytics layer (rather than the batch warehouse), probably displays live traffic, transit, and weather overlays on a map interface for city operators, with access controls limiting which roles can see which data layers.

Q5. Healthcare Analytics System: Records, Wearables, Labs & Appointments

A healthcare analytics system integrating patient records, wearable device readings, lab reports, and appointment histories has most likely been built around ETL pipelines built on healthcare interoperability standards, since the data originates from multiple independent hospital/clinic systems that would otherwise be incompatible.

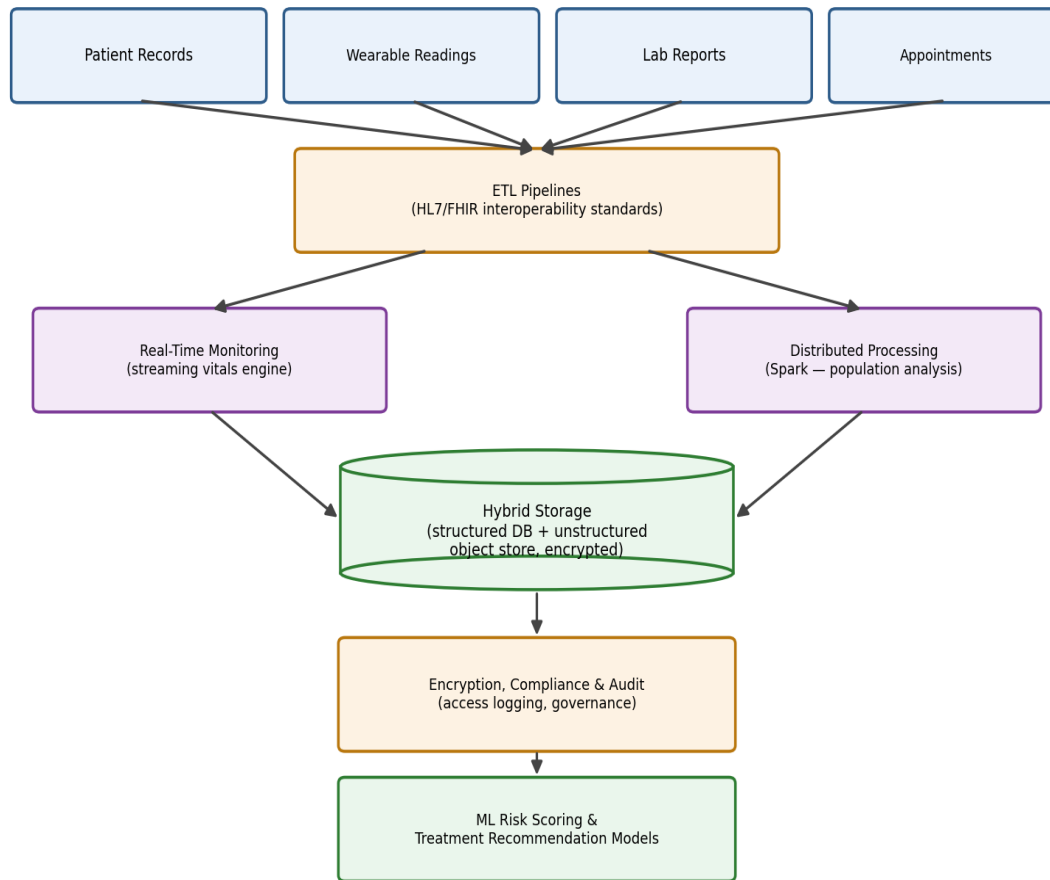
Probable hybrid storage architecture

Similar to other healthcare systems, structured data (patient records, appointment histories, lab report metadata) most likely lives in a relational or structured NoSQL store, while larger or less-structured content (raw lab report documents, wearable time-series dumps) most likely lives in object storage — combined into a hybrid architecture so each data type is stored in the system best suited to it, with encryption applied uniformly across both.

Likely ETL pipelines and interoperability standards

Given that hospitals and clinics are named explicitly, ETL pipelines built around HL7/FHIR standards are highly probable, extracting data from each source system's native format, transforming it into a common patient-centric schema, and loading it into the unified analytics store — this is the only practical way to reconcile data from institutions running different, often legacy, systems.

Q5 — Reverse-Engineered Healthcare Analytics System



Real-time monitoring and predictive health analytics implementation

Wearable device readings most likely flow through a streaming pipeline for real-time monitoring (e.g., detecting abnormal heart rate patterns as they occur), while predictive health analytics (e.g., disease risk scores) most likely run as periodically retrained ML models applied to the combined structured and streaming data — reflecting that training complex predictive models is computationally expensive and doesn't need to happen per-event the way monitoring alerts do.

Role of distributed processing frameworks in population-level analysis

A distributed processing framework (most likely Apache Spark) is probable for population-level analysis, since answering questions like "what factors predict readmission across all patients" requires scanning and aggregating enormous historical datasets that a single machine could not process in reasonable time.

Encryption, compliance, and audit mechanisms

Given the sensitivity of medical data spanning multiple institutions, the system most likely enforces end-to-end encryption (at rest and in transit), strict role-based and possibly attribute-based access control, and comprehensive audit logging of every data access — required for compliance with health data regulations — with governance likely including data-sharing agreements between the participating hospitals and clinics defining exactly what can be accessed and by whom.

Machine learning for diagnosis risk scoring and treatment recommendations

Diagnosis risk-scoring models are most likely supervised classification models trained on historical labeled outcomes (e.g., which patients with similar profiles were later diagnosed with a condition), while treatment recommendation likely draws on a combination of clinical guidelines encoded as rules and a learned model trained on outcomes from similar past cases — with a human clinician almost certainly kept in the loop as the final decision-maker given the high stakes of medical recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented